

6장 액티베이션 레코드

최광훈
전남대학교

액티베이션 레코드

핵심 주제: 함수 호출 방법과 스택 프레임 구조

- 예: Java 메소드

```
int f(int x) {  
    int y = x+x;  
    if (y<10)  
        return f(y);  
    else  
        return y-1;  
}
```

함수에 진입할 때 지역 변수를 새로 만듦
- f(int x)를 (재귀) 호출할 때마다
새로운 x와 y 변수 생성

함수에서 리턴할 때 지역 변수를 없앴
- x와 y 변수를 없앴

함수 호출은 후입선출(LIFO) 방식으로 동작
- LIFO 특성 때문에 지역 변수를 저장하기 위해
스택 자료 구조를 사용 가능

액티베이션 레코드

스택 자료 구조로 구현하지 못하는 유형의 함수들

디스플레이(display)

- 중첩 함수(nested functions): 중첩 함수 지역 변수 참조를 위한 별도 자료 구조 필요
- 고차원 함수(higher-order functions): 지역 변수를 힙에 저장

```
fun f(x) =  
  let fun g(y) = x+y  
    in g  
  end
```

```
val h = f(3)  
val j = f(4)
```

```
val z = h(5)  
val w = j(7)
```

(a) Written in ML

```
int (*)() f(int x) {  
  int g(int y) {return x+y;}  
  return g;  
}
```

```
int (*h)() = f(3);  
int (*j)() = f(4);
```

```
int z = h(5);  
int w = j(7);
```

(b) Written in pseudo-C

PROGRAM 6.1. An example of higher-order functions.

[프로그램 6.1]

중첩 함수와 함수 값 변수를 모두 지원하는 언어에서는 함수가 반환된 후에도 로컬 변수를 유지해야 할 필요가 있을 수 있다!

프로그램 6.1을 고려해 보자. ML에서는 중첩 함수와 함수 값 변수 모두 합법적이지만, C에서는 함수 f 안에 함수 g 를 실제로 중첩할 수 없다.

$f(3)$ 이 실행될 때, 함수 f 의 활성화에 사용될 새로운 지역 변수 x 가 생성된다. 그런 다음 $f(x)$ 의 결과로 g 가 반환되지만, g 는 아직 호출되지 않았으므로 y 는 아직 생성되지 않았다.

이 시점에서 f 는 반환되었지만, x 를 파괴하기에는 너무 이르다. 왜냐하면 $h(5)$ 가 결국 실행될 때 $x = 3$ 의 값이 필요하기 때문이다.

한편, $f(4)$ 가 호출되어 x 의 다른 인스턴스를 생성하고, $x = 4$ 인 g 의 다른 인스턴스를 반환한다.

중첩 함수(내부 함수가 외부 함수에서 정의된 변수를 사용할 수 있음)와 결과로 반환되거나 변수에 저장되는 함수의 조합으로 인해, 지역 변수는 이를 호출하는 함수보다 긴 수명이 필요하게 된다.

파스칼은 중첩 함수를 지원하지만 함수를 반환 가능한 값으로 제공하지 않는다. C는 함수를 반환 가능한 값으로 제공하지만 중첩 함수는 지원하지 않는다. 따라서 이 언어들은 스택을 사용하여 지역 변수를 저장할 수 있다.

ML, Scheme 및 여러 다른 언어들은 중첩 함수와 반환 가능한 함수(이 조합을 고차 함수라고 함)를 모두 지원한다. 따라서 스택으로 모든 지역 변수를 저장할 수 없다. 이는 ML과 Scheme의 구현을 복잡하게 만듭니다. 그러나 고차 함수가 제공하는 추가적인 표현력이 크기 때문에 이 추가적인 구현에 노력을 들인다.

액티베이션 레코드

핵심 주제: 함수 호출 방법과 스택 프레임 구조

- Java는 중첩 함수를 지원하지만 (중첩 클래스를 통해서)
 - MiniJava는 스택으로 쌓을 수 있는 지역 변수만 지원
- => 스택으로 함수 지역 변수 생성 및 해제를 관리

6.1 스택 프레임

스택 처리 방식 (배열 모델)

- 스택을 하나의 큰 배열로 취급, 스택 포인터(SP)라는 특수 레지스터 사용
- SP 이전 위치는 할당된 영역, SP 이후 위치는 할당되지 않은(가비지) 영역
- 스택은 함수 진입 시 충분히 크게 확장되고, 종료 직전에 동일하게 축소됨

활성화 레코드(스택 프레임) 정의

- 함수의 지역 변수, 매개변수, 반환 주소 및 기타 임시 변수를 위한 스택 영역

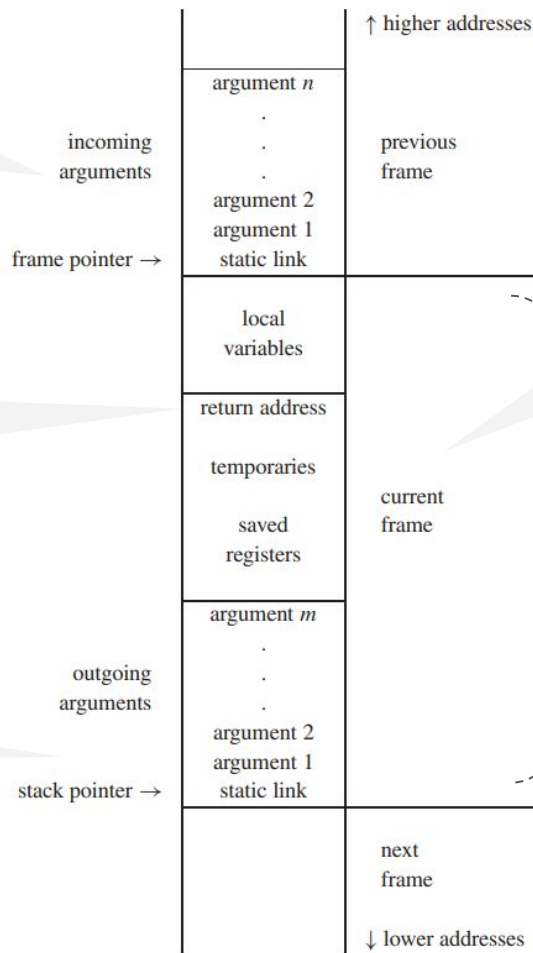
메모리 확장 방향

- 런타임 스택은 일반적으로 높은 메모리 주소에서 낮은 주소 방향으로 확장

프레임에는 호출자가 전달한 일련의 인자(기술적으로는 이전 프레임의 일부이지만 프레임 포인터로부터 알려진 오프셋에 위치함)

반환 주소는 CALL 명령어에 의해 생성됨 (현재 함수 완료 시 제어권이 호출 함수 내 어디로 돌아가야 하는지 알려주는 역할)

중첩된 함수를 구현할 때 static link 관리 필요



스택 프레임에 저장되는
임시 변수

Spilled 레지스터 저장 공간

FIGURE 6.2. A stack frame.

6.1 스택 프레임

프레임 레이아웃 설계 요소

- 명령어 집합 아키텍처(ISA)
- 컴파일 대상 프로그래밍 언어의 특성

표준 레이아웃의 중요성

- 컴퓨터 제조사는 가능한 경우 모든 컴파일러가 사용할 표준 프레임 레이아웃을 규정
- 한 언어로 작성된 함수가 다른 언어로 작성된 함수를 호출할 수 있음

6.1 스택 프레임

지역변수 저장 위치

- 일부 지역변수는 스택 프레임(메모리)에 위치하고,
- 다른 지역변수는 레지스터에 보관되어 빠르게 읽고 쓰기함

레지스터: CPU 안에 위치한 메모리
- 기계어 명령어의 피연산자로 사용

스필링(Spilling)

- 레지스터에 보관된 지역변수는, 레지스터를 다른 용도로 사용하기 위해 공간을 확보해야할 때 프레임에 저장되어야 함
 - 프레임 내에 이러한 목적(저장된 레지스터 및 임시 변수)을 위한 별도 영역이 있음
- 함수 호출시 전달할 인자들(outgoing arguments)을 위한 공간

6.1 스택 프레임

프레임 포인터(FP)

- 함수 g (호출자)가 함수 f (피호출자)를 호출할 때
- f 에 진입할 때 스택 포인터(SP)에서 프레임 크기 만큼 프레임을 할당

FP의 정의

- 기존 SP는 현재 FP가 됨
- f 를 종료할 때 FP를 SP로 복사하고 저장된 FP를 다시 불러옴
- 프레임 크기가 변동하거나 스택에서 프레임이 항상 연속적이지 않을 때 FP가 유용함

6.1 스택 프레임

프레임 크기는 컴파일 과정이 상당히 진행된 후, 즉 메모리에 상주하는 임시 변수와 저장된 레지스터의 수가 결정될때에야 비로소 결정됨.

하지만 형식 매개변수와 지역변수의 오프셋은 훨씬 일찍 알 필요가 있음.

따라서 편의상 프레임 포인터를 다루고자 함.

형식 매개변수와 지역변수는 일찍 알려진 오프셋(FP에 바로 근처)에 배치되고,

임시변수와 저장된 레지스터들은 나중에 알려진 오프셋에 배치됨

6.1 스택 프레임

레지스터

- 컴파일된 프로그램의 속도를 높이기 위해 지역변수, 식의 중간 결과 등을 스택 프레임 대신 레지스터에 보관
- 산술 명령어로 직접 접근 가능
- 대부분의 CPU에서 메모리 접근은 별도의 load 및 store 명령어가 필요하므로 레지스터 접근이 더 빠름

Register name	Number	Usage
\$zero	0	constant 0
\$at	1	reserved for assembler
\$v0	2	expression evaluation and results of a function
\$v1	3	expression evaluation and results of a function
\$a0	4	argument 1
\$a1	5	argument 2
\$a2	6	argument 3
\$a3	7	argument 4
\$t0	8	temporary (not preserved across call)
\$t1	9	temporary (not preserved across call)
\$t2	10	temporary (not preserved across call)
\$t3	11	temporary (not preserved across call)
\$t4	12	temporary (not preserved across call)
\$t5	13	temporary (not preserved across call)
\$t6	14	temporary (not preserved across call)
\$t7	15	temporary (not preserved across call)
\$s0	16	saved temporary (preserved across call)
\$s1	17	saved temporary (preserved across call)
\$s2	18	saved temporary (preserved across call)
\$s3	19	saved temporary (preserved across call)
\$s4	20	saved temporary (preserved across call)
\$s5	21	saved temporary (preserved across call)
\$s6	22	saved temporary (preserved across call)
\$s7	23	saved temporary (preserved across call)
\$t8	24	temporary (not preserved across call)
\$t9	25	temporary (not preserved across call)
\$k0	26	reserved for OS kernel
\$k1	27	reserved for OS kernel
\$gp	28	pointer to global area
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	return address (used by function call)

```

.text
.align 2
.globl main

main:
    subu    $sp, $sp, 32
    sw      $ra, 20($sp)
    sd      $a0, 32($sp)
    sw      $0, 24($sp)
    sw      $0, 28($sp)

loop:
    lw      $t6, 28($sp)
    mul     $t7, $t6, $t6
    lw      $t8, 24($sp)
    addu    $t9, $t8, $t7
    sw      $t9, 24($sp)
    addu    $t0, $t6, 1
    sw      $t0, 28($sp)
    ble     $t0, 100, loop
    la      $a0, str
    lw      $a1, 24($sp)
    jal     printf
    move    $v0, $0
    lw      $ra, 20($sp)
    addu    $sp, $sp, 32
    jr      $ra

.data
.align 0

str:
    .asciiz "The sum from 0 .. 100 is %d\n"

```

FIGURE A.6.1 MIPS registers and usage convention.

6.1 스택 프레임

레지스터 보존 규칙

- Caller-Save vs Callee-Save
- 하나의 레지스터 셋을 여러 함수가 공유, 함수 호출 시 레지스터 충돌을 피해야 함.
- 함수 f 와 함수 g 가 모두 레지스터 r 을 사용한다면, f 가 g 를 호출할 때
 g 가 r 을 사용하기 전에 f 가 사용하던 r 을 저장하고 g 를 마친 다음 복원해야 함

레지스터 보존 책임 분담

- 호출자 저장(Caller-Save): 레지스터 저장과 복원의 책임을 호출자(f)에게 부여
- 피호출자 저장(Callee-Save): 레지스터 저장과 복원의 책임을 피호출자(g)에게 부여

6.1 스택 프레임

예를 들어 MIPS 컴퓨터에서는

- 레지스터 16~23이 프로시저 호출 간에 보존되며(피호출자 보존),
- 그 외 모든 레지스터는 프로시저 호출 간에 보존되지 않음(호출자 보존).

(참고: Assemblers, Linkers, and the SPIM Simulator,

A.6절 MIPS CPU 레지스터 설명)

6.1 스택 프레임

각 지역 변수와 임시 변수에 대해 호출자 저장 또는 피호출자 저장 레지스터를 현명하게 선택하면 프로그램이 실행하는 저장(store) 및 가져오기(fetch) 횟수를 줄일 수 있음.

레지스터 저장 및 복원 최적화

컴파일러의 레지스터 할당기는 적합한 종류의 레지스터를 선택하는 역할을 담당함

- 저장이 불필요한 경우:

함수 f 가 변수 x 의 값이 호출 후 필요하지 않음을 알 경우,

x 를 호출자 저장 레지스터에 보관하고 g 호출 시 저장하지 않음

- 저장을 최소화하는 경우:

함수 f 가 여러 함수 호출 전후에 필요한 지역변수 i 를 가질 경우,

f 진입 시 단 한 번만 i 를 피호출자 저장 레지스터 r 에 저장하고 (load)

f 반환 전에 단 한 번만 r 에서 i 로 다시 가져올 수 있음(save)

6.1 스택 프레임

인자 전달(Parameter Passing)

- 과거(1970년대): 함수 인자는 스택을 통해 전달하여 불필요한 메모리 트래픽을 유발
- 현대: 첫 k개 인수는 레지스터로 전달하고, 나머지는 메모리로 전달

레지스터 사용이 시간을 절약하는 이유

- 다른 함수를 호출하지 않는 함수
- 프로시저간 레지스터 할당
- 더이상 사용하지 않는 변수
- 레지스터 윈도우(register windows) : SPARC 아키텍처

6.1 스택 프레임

인자를 레지스터에 저장해서 전달하기 어렵게 만듦

탈출 (Escape) 변수

- 예) C언어
- 형식 매개변수의 주소를 취하는 것을 허용
- 함수의 모든 형식 매개변수가 연속된 주소에 위치 (printf의 varargs 기능)
- 'dangling reference' : 예) `int* f() { int x=1; return &x; }`

6.1 스택 프레임

리턴 주소

- 주소 a 에서 함수 g 가 f 를 호출한 다음 g 로 돌아갈 때 명령어 주소는 $a+1$ (함수 호출 다음 명령어)

리턴 주소 전달 방식

- 과거 방식(1970년대): call 명령어가 리턴 주소를 스택에 push
- 현대 방식: 리턴 주소를 지정된 레지스터로 전달
- 메모리 트래픽을 피하고 하드웨어에 특정 스택 규칙을 강제하지 않음
- Non-leaf 함수는 레지스터에 있는 리턴 주소를 메모리에 저장해야 함

6.1 스택 프레임

변수(값)를 레지스터 대신 스택 프레임(메모리)에 기록하는 경우

- 주소 참조가 필요한 경우: 참조 전달, &연산자, 중첩된 함수의 변수 접근

 - => 탈출 변수(escape variables)

- 데이터 크기/구조 문제: 값이 너무 클 때, 배열 변수

- 지역 변수와 임시 값이 너무 많아 레지스터가 부족할 때 (spill), 매개변수 전달과

 - 같은 특수한 목적으로 레지스터를 사용할 때

산업용 컴파일러는 모든 형식 매개변수와 지역 변수에 임시 위치를 할당한 후, 나중에 실제로 레지스터에 배치해야할 대상을 결정

6.1 스택 프레임

정적 링크(static links)

- 함수 f 가 호출될 때마다, f 를 프로그램 텍스트 내에서 바로 둘러싸는 함수 g 의 현재 (가장 최근에 진입한) 스택 프레임에 대한 포인터가 f 에 전달됨. 이 포인터가 정적 링크
- 정적 링크를 사용하여 비지역 변수를 접근
(체인의 길이: 두 함수 간의 정적 중첩 깊이 차이)

정적 링크의 대안

- 디스플레이(display)
- 람다 리프팅(lambda lifting)

[실습] MIPS 어셈블리 프로그램 실행하기

- James R. Larus, Assemblers, Linkers,
and the SPIM Simulator

- Figure A.1.3: 0~100까지의 합 구하기

- MARS 4.1 시뮬레이터
(Jens/kanna/mars.jar)

```
java -jar mars.jar
- File -> Open -> prog.mips
- Run -> Assemble
- Run -> Go
```

- <https://dpetersanderson.github.io/>

FIGURE A.1.4 The same routine written in assembly language with labels, but no comments. The commands that start with periods are assembler directives (see pages A-47–A-49). `.text` indicates that succeeding lines contain instructions. `.data` indicates that they contain data. `.align n` indicates that the items on the succeeding lines should be aligned on a 2^n byte boundary. Hence, `.align 2` means the next item should be on a word boundary. `.globl main` declares that `main` is a global symbol that should be visible to code stored in other files. Finally, `.asciiz` stores a null-terminated string in memory.

```
.text
.align 2
.globl main

main:
    subu    $sp, $sp, 32
    sw      $ra, 20($sp)
    sd      $a0, 32($sp)
    sw      $0, 24($sp)
    sw      $0, 28($sp)

loop:
    lw      $t6, 28($sp)
    mul     $t7, $t6, $t6
    lw      $t8, 24($sp)
    addu    $t9, $t8, $t7
    sw      $t9, 24($sp)
    addu    $t0, $t6, 1
    sw      $t0, 28($sp)
    ble     $t0, 100, loop
    la      $a0, str
    lw      $a1, 24($sp)
    jal     printf
    move    $v0, $0
    lw      $ra, 20($sp)
    addu    $sp, $sp, 32
    jr      $ra

.data
.align 0

str:
.asciiz "The sum from 0 .. 100 is %d\n"
```

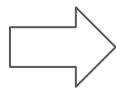
6.2 MiniJava 컴파일러의 스택 프레임

소스 프로그램 => IR 프로그램 => MIPS 어셈블리

소스 프로그램 => IR 프로그램 (7장)

```
int main() {  
    int sum = 0;  
  
    for (int i = 0; i <= 100; i++) {  
        sum = sum + i * i;  
    }  
  
    _print("The sum from 0 .. 100 is %d\n", sum);  
  
    return 0;  
}
```

어셈블리 수준의 명령어에
매핑될 수 있는
한가지 연산으로만 구성된
식 또는 문장으로 변환



```
int main() {  
    int sum;  
    int i;  
  
    sum = 0;  
    i = 0;  
  
loop:  
    {  
        int t6 = i;  
        int t7 = t6 * t6;  
        int t8 = sum;  
        int t9 = t8 + t7;  
  
        sum = t9;  
  
        int t0 = t6 + 1;  
        i = t0;  
  
        if (t0 <= 100)  
            goto loop;  
    }  
    _print("The sum from 0 .. 100 is %d\n",  
          sum);  
    return 0;  
}
```


IR 프로그램 => MIPS 어셈블리

```
int main() {  
    int sum;  
    int i;
```

```
    sum = 0;  
    i = 0;
```

```
loop:  
{  
    int t6 = i;  
    int t7 = t6 * t6;  
    int t8 = sum;  
    int t9 = t8 + t7;
```

```
    sum = t9;
```

```
    int t0 = t6 + 1;  
    i = t0;
```

```
    if (t0 <= 100)  
        goto loop;
```

```
}  
_print("The sum from 0 .. 100 is %d\n", sum);  
return 0;
```

```
}
```

1. 명령어 선택 (9장)

t7 = t6 * t6
=> mul \$t7, \$t6, \$t6

2. 레지스터 할당 (11장)

[레지스터 할당 전]

IR 프로그램

임시 변수 이름

t101, t102, t103, t104,

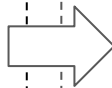
[레지스터 할당 후]

t101 => \$t6

t102 => \$t7

t103 => \$t8

t104 => \$t9



```
.text  
.align    2  
.globl    main  
  
main:  
    subu    $sp, $sp, 32  
    sw      $ra, 20($sp)  
    sd      $a0, 32($sp)  
    sw      $0, 24($sp)  
    sw      $0, 28($sp)  
  
loop:  
    lw      $t6, 28($sp)  
    mul     $t7, $t6, $t6  
    lw      $t8, 24($sp)  
    addu    $t9, $t8, $t7  
    sw      $t9, 24($sp)  
    addu    $t0, $t6, 1  
    sw      $t0, 28($sp)  
    ble     $t0, 100, loop  
    la      $a0, str  
    lw      $a1, 24($sp)  
    jal     printf  
    move    $v0, $0  
    lw      $ra, 20($sp)  
    addu    $sp, $sp, 32  
    jr      $ra
```

```
.data  
.align    0  
  
str:  
    .asciiz "The sum from 0 .. 100 is %d\n"
```

6.2 MiniJava 컴파일러의 스택 프레임

소스 언어 변수와 MIPS 어셈블리 레지스터 매핑

소스 코드	MIPS 어셈블리	설명
반환 주소 저장	20(\$sp)	\$ra 저장 위치
sum	24(\$sp)	제곱들의 누적 합
i	28(\$sp)	반복 변수

6.2 MiniJava 컴파일러의 스택 프레임

임시 계산 결과와 MIPS 레지스터 매핑

레지스터	소스 프로그램	대응 어셈블리
\$t6	현재 i 값	lw \$t6, 28(\$sp)
\$t7	$i \cdot i$ 제곱값	mul \$t7, \$t6, \$t6
\$t8	현재 sum 값	lw \$t8, 24(\$sp)
\$t9	$\text{sum} + \text{square}$	addu \$t9, \$t8, \$t7
\$t10	$i + 1$	addu \$t0, \$t6, 1

6.2 MiniJava 컴파일러의 스택 프레임

MiniJava 컴파일러가 대상 아키텍처의 특정 스택 프레임 레이아웃에 의존하지 않도록

- Frame을 추상 인터페이스로 구성

name: 함수 레이블 이름
formals: 형식 매개변수 목록(k accesses)
allocLocal(boolean escapes): 지역 변수 할당

```
public abstract class Frame {  
    abstract public Frame newFrame(Label name,  
                                    Util.BoolList formals);  
  
    public Label name;  
    public AccessList formals;  
    abstract public Access allocLocal(boolean escape);  
    /* ... other stuff, eventually ... */  
}
```

6.2 MiniJava 컴파일러의 스택 프레임

MiniJava 컴파일러가 대상 아키텍처의 특정 스택 프레임 레이아웃에 의존하지 않도록

- Access: 형식 매개변수 및 지역 변수의 저장 위치

```
package Frame;  
import Temp.Temp; import Temp.Label;  
  
public abstract class Access { ... }  
public abstract class AccessList {...head;...tail;... }
```

6.2 MiniJava 컴파일러의 스택 프레임

대상 아키텍처에 의존하는 Frame 구현

```
package Mips;
class Frame extends Frame.Frame { ... }
```

```
// in class Main.Main:
```

```
Frame.Frame frame = new Mips.Frame(...);
```

[illegible]

6.2 MiniJava 컴파일러의 스택 프레임

변수가 스택에서 ‘escape’

- 현재 함수의 실행이 끝난 뒤에도

그 함수 지역변수에 접근할 가능성이 있음

- 변수의 생명주기가 접근 범위가 현재 함수 호출의

스택 프레임을 벗어남

```
int* f() {  
    int x = 10;  
    return &x;  
}
```

```
class A {  
    int f() {  
        int x = 10;  
  
        class Inner {  
            int g() {  
                return x;  
            }  
        }  
  
        return new Inner().g();  
    }  
}
```

6.2 MiniJava 컴파일러의 스택 프레임

Frame 구현

- 모든 형식 매개변수의 위치, 뷰 전환을 구현하는 명령어, 할당한 지역 변수 갯수, 함수 코드 라벨

MIPS calling convention은 인자를 r4, r5, r6로 넘기지만, 컴파일러 내부에서는 각 formal parameter를 InFrame 또는 InReg로 관리한다.

K는 스택 프레임의 크기.

Shift of view는 이 두 관점 사이를 맞추는 함수 시작부의 이동 코드이다.

	Pentium	MIPS	Sparc
Formals	1 InFrame(8)	InFrame(0)	InFrame(68)
	2 InFrame(12)	InReg(t_{157})	InReg(t_{157})
	3 InFrame(16)	InReg(t_{158})	InReg(t_{158})
View Shift	$M[sp + 0] \leftarrow fp$	$sp \leftarrow sp - K$	save %sp, -K, %sp
	$fp \leftarrow sp$	$M[sp + K + 0] \leftarrow r4$	$M[fp + 68] \leftarrow i0$
	$sp \leftarrow sp - K$	$t_{157} \leftarrow r5$ $t_{158} \leftarrow r6$	$t_{157} \leftarrow i1$ $t_{158} \leftarrow i2$

TABLE 6.4.

Formal parameters for $g(x_1, x_2, x_3)$ where x_1 escapes.

6.2 MiniJava 컴파일러의 스택 프레임

호출 함수와 피호출 함수에 따라 매개변수를 다르게 인식(Shift of View)

스택 전달 예시

- 호출 함수: 스택 포인터에서 오프셋 4 위치에 매개변수를 배치
- 피호출 함수: 프레임 포인터에서 오프셋 4 위치에서 이를 인식

레지스터 전달 예시

- 호출 함수: 인자를 레지스터 o1에 저장
- 피호출 함수: 레지스터 i1을 통해 매개변수를 접근

6.2 MiniJava 컴파일러의 스택 프레임

함수의 매개변수들은 호출 규칙에 따라 처음에는 특정 레지스터나 스택 위치에 들어오지만, 함수 내부에서는 컴파일러가 정한 방식으로 접근해야 한다. 이 차이를 맞추는 작업이 Shift of View

- $r4, r5, r6 \Rightarrow M[SP+K+0], t157, t158$

Frame.newFrame은 각 형식 매개변수에 대해 다음 두 가지를 담당

- 함수 내부 인식 방식: 매개변수가 레지스터에, 아니면 프레임 위치에 저장되는지
- 뷰 전환 명령어: 뷰 전환을 구현하기 위해 생성해야 할 명령어

6.2 MiniJava 컴파일러의 스택 프레임

지역 변수 할당 방법

- 프레임 `frame.allocLocal(false)`

인자 `false` : escape 변수가 아님. 레지스터에 할당 가능
반환 값 Access 객체 : 할당 위치

- 프레임 내: `InFrame(-4)`
- 레지스터 내: `InReg(t481)`

MiniJava에서는 변수가 프레임 밖으로 탈출하지 않음

- 클래스와 메서드의 중첩이 존재하지 않고
- 변수의 주소를 취할 수 없으며
- 정수와 부울은 값으로 전달
- 객체와 정수 배열은 포인터로 표현되며, 이 포인터 역시 값으로 전달

6.2 MiniJava 컴파일러의 스택 프레임

지역 변수 할당 방법

- frame.allocLocal(false) 호출은 프레임 생성 직후에 이루어질 필요가 없음
- 함수 내 3가지 다른 변수 v에 대해서 프레임 내에서 서로 다른 위치를 할당
- 레지스터 할당 최적화나 프레임 슬롯 최적화 가능성

```
void f()  
{int v=6;  
  print(v);  
  {int v=7;  
    print(v);  
  }  
  print(v);  
  {int v=8;  
    print(v);  
  }  
  print(v);  
}
```

6.2 MiniJava 컴파일러의 스택 프레임

임시 변수(Temps) 및 레이블(Labels)

- new Temp.Temp(): 새로운 임시 변수
- new Temp.Label(): 새로운 레이블
- new Temp.Label(string):
- 예) new Temp.Label("C"+"\$"+"m")

```
package Temp;
public class Temp {
    public String toString();
    public Temp();
}
public class Label {
    public String toString();
    public Label();
    public Label(String s);
    public Label(Symbol s);
}
public class TempList {...}
public class LabelList {...}
```

6.2 MiniJava 컴파일러의 스택 프레임

임시 변수(Temps) 및 레이블(Labels)

개념	정의	용도
임시 변수(temporary)	레지스터에 일시적으로 보관되는 값에 대한 추상 이름	지역 변수를 위한 추상 이름
레이블(label)	정확한 주소가 아직 결정되지 않은 기계어 위치에 대한 추상 이름(어셈블리어 레이블)	함수 본문과 같은 정적 메모리 주소를 위한 추상 이름

- 컴파일러의 번역 단계에서 매개변수와 지역 변수를 위한 레지스터를 선택하고 함수 본문을 위한 기계어 주소를 선택해야 하지만, 정확한 레지스터 가용성이나 함수 위치를 일찍 결정할 수 없어 추상 이름을 사용

6.2 MiniJava 컴파일러의 스택 프레임

정적 링크 관리

- MiniJava는 중첩 함수 선언을 지원하지 않으므로
- Frame 모듈에서 정적 링크에 대해 고려할 사항이 없음

6.2 MiniJava 컴파일러의 스택 프레임

런타임 함수 호출: `Frame.externalCall`

- 외부/런타임 함수 호출은 CPU/OS별로 다름

예) `initArray`, `_print`, `_alloc`

- `Frame` 클래스서 추상화 메소드 정의

```
public abstract class Frame {  
    abstract public Tree.Exp externalCall(  
        String func,  
        Tree.ExpList args  
    );  
}
```


[실습] MiniJava 컴파일러의 프레임

MIPS를 타겟으로 MiniJava 컴파일러를 만들고자 한다.

- Mips 패키지: Mips/Access.java, Mips/Frame.java

Mips.Frame

- String name
- List<Access> formals
- frameSize
- WORD=4, K (k개 인자는 레지스터로, 나머지는 메모리로 인자 전달)